

INSTITUTO
FEDERAL
Santa Catarina

Resumo de matérias

Pensamento Computacional e Algorítmos

Eduardo Cardoso Oliveira, 2025

Aviso importante e Isenção de responsabilidade

1. **Finalidade do documento**
   Este documento foi elaborado com o objetivo de servir como um material de apoio e revisão para os estudos das matérias do primeiro semestre de Análise e Desenvolvimento de Sistemas. Todos os conteúdos aqui presentes foram retirados de anotações pessoais e resumos, compilados para facilitar a compreensão do leitor.

2. **Fontes e Direitos Autorais**
   Os resumos e anotações aqui presentes foram desenvolvidos a partir dos materiais de sala disponibilizados pelos professores (como slides, apostilas, listas de exercícios, livros, etc), anotações de aula e pesquisas. Os direitos autorais sobre os materiais originais (aulas, livros, artigos) permanecem integralmente aos seus respectivos autores e editoras. Este documento é uma obra derivada, de caráter estudantil e sem fins lucrativos.

3. **Precisão das Informações**
   Desde já, o autor não oferece nenhuma garantia de que os conteúdos aqui contidos sejam precisos e corretos, por mais que todo esforço tenha sido feito para garantir que tais informações sejam corretas. Erros, omissões ou interpretações equivocadas podem existir. O autor não se responsabiliza por quaisquer consequências, acadêmicas ou de outra natureza, que possam surgir do uso deste material.

4. **Uso Recomendado**
   Este material **NÃO SUBSTITUI DE FORMA ALGUMA** a leitura de bibliografias recomendadas por professores, presença em aulas, consulta às fontes oficiais e, principalmente, as orientações do corpo docente. Utilize este documento como uma ferramenta complementar aos seus estudos.

5. **Vínculo Institucional**
   Este é um projeto independente e não possui qualquer vínculo oficial com o Instituto Federal de Santa Catarina Campus São José, a Coordenação do Curso de Análise e Desenvolvimento de Sistemas, ou os professores das disciplinas. Não há qualquer endosso ou revisão oficial do conteúdo por parte da instituição.

Apresentação do Material

Olá, colega! Seja bem-vindo(a) ao meu resumo de estudos! Este documento foi criado ao longo do semestre com o objetivo de centralizar e facilitar o acesso aos conteúdos que foram abordados durante as aulas do primeiro semestre do curso em 2025.1 na disciplina de Pensamento Computacional e Algorítmos.

O que você vai encontrar neste documento?

- **Resumos:** Para esta disciplina trago resumo de slides, levantando pontos que considero os mais importantes.
- **Ferramentas:** Também trago algumas recomendações de ferramentas para utilizar aqui, que ajudam no aprendizado e organização.
- **Documentos e bibliografias:** Quando possível, vou disponibilizar materiais, resumos, listas de exercícios, gabaritos e referências bibliográficas para auxiliar.

Como este documento está organizado?

O material foi dividido por disciplinas, por ordem de andamento do semestre 2025.1. Cada sessão é um bloco de conteúdos vistos em sala de aula.

Informações adicionais e contato

Este material foi feito de aluno para aluno; sugestões e colaborações são sempre bem-vindas! Se encontrar algum erro, tiver alguma sugestão para adicionar ao documento ou tirar alguma dúvida sobre qualquer matéria, fique à vontade para entrar em contato.

- **Autor:** Eduardo Cardoso Oliveira
- **E-mail Institucional:** eduardo.co22@aluno.ifsc.edu.br
- **Github:** https://github.com/EduCardoso22

Sumário

1 Apresentação da Disciplina **4**
1.1 Conceitos Fundamentais 4
1.2 Importância de Aprender a Programar 4
1.3 Linguagens e Ferramentas do Curso 4

2 Introdução ao Pensamento Computacional **5**
2.1 Os Quatro Pilares do Pensamento Computacional 5

3 Introdução aos Algoritmos **6**
3.1 Descrição Narrativa 6
3.2 Fluxograma 6
3.3 Pseudocódigo 6

4 Manual de Pseudocódigo **7**
4.1 Estrutura Básica de um Algoritmo 7
4.2 Variáveis e Tipos de Dados 7
4.3 Comandos de Entrada e Saída 7
4.4 Operadores 8
4.5 Estruturas de Controle 8

5 Introdução à Programação Orientada a Objetos (POO) **9**
5.1 Conceitos Fundamentais da POO (Analogia com Lego) 9
5.2 Propósito da POO 9
5.3 Linguagem e Ferramentas 9

6 Manual de Sintaxe da Linguagem Java **10**
6.1 Estrutura Fundamental e Tipos de Dados 10
6.2 Entrada e Saída de Dados (I/O) 10
6.3 Classes Wrapper 10
6.4 Operadores 11
6.5 Estruturas de Controle 11
6.6 Arrays 11
6.7 Manipulação de Strings 11

7 Referências **13**

1 Apresentação da Disciplina

1.1 Conceitos Fundamentais

- **Pensamento Computacional**: Uma abordagem para formular e resolver problemas complexos, dividindo-os em etapas simples. É baseado em quatro pilares: Decomposição, Reconhecimento de Padrões, Abstração e Algoritmos.
- **Algoritmo**: Uma sequência lógica e bem definida de passos para resolver um problema ou realizar uma tarefa.
- **Programação**: O processo de escrever instruções para um computador executar tarefas, funcionando como uma forma de comunicação entre humanos e máquinas.

1.2 Importância de Aprender a Programar

Aprender a programar é importante por diversos motivos, incluindo:

- Desenvolver habilidades cognitivas como pensamento lógico e resolução de problemas.
- Atender à alta demanda do mercado de trabalho.
- Ter aplicação em diversas áreas, como IA, games e robótica.

1.3 Linguagens e Ferramentas do Curso

- A disciplina, neste semestre, usou principalmente a linguagem **Java**, juntamente com **Pseudocódigo** (baseado em Portugol) e **Fluxogramas**.
- A escolha do Java se deve à sua vasta aplicação em sistemas para nuvem, mobile, desktop, web e embarcados. A linguagem é destacada por sua alta portabilidade e gerenciamento de memória automático.
- **Ferramentas a serem utilizadas**:
  - **PseudocodeLab**: Interpretador de pseudocódigo.
  - **FluxoLab**: Ferramenta para criação e execução de fluxogramas.
  - **Github Codespace e VSCode**: Para compilação de código Java.

2 Introdução ao Pensamento Computacional

Teremos aqui uma introdução sobre o Pensamento Computacional como um processo para formular problemas e suas soluções de forma que possam ser eficientemente executadas por humanos ou computadores. O Pensamento Computacional é destacado como uma habilidade essencial do século XXI, que prepara para o mercado de trabalho moderno ao valorizar a inovação e a capacidade de resolver problemas complexos. Sua aplicação vai além da computação, sendo útil em matemática, ciências, economia e em diversas tarefas do cotidiano, como planejar uma viagem, seguir uma receita ou organizar uma rotina de estudos.

2.1 Os Quatro Pilares do Pensamento Computacional

O Pensamento Computacional é fundamentado em quatro pilares principais, que são aplicados para lidar com problemas complexos:

1. **Decomposição**:
   - *Definição*: Consiste em quebrar problemas complexos em partes menores, mais simples e gerenciáveis.
   - *Importância*: Facilita o entendimento do problema, permite a resolução de uma parte de cada vez, auxilia na delegação de tarefas e possibilita a reutilização de soluções. Um exemplo prático é dividir a tarefa de organizar uma mochila em subtarefas, como separar livros, lanche e outros materiais.
2. **Reconhecimento de Padrões**:
   - *Definição*: É a habilidade de identificar semelhanças, tendências e regularidades nos dados ou dentro das partes decompostas de um problema.
   - *Importância*: Simplifica a resolução de problemas, ajuda a prever comportamentos futuros com base em experiências passadas e facilita a reutilização de soluções conhecidas. Ao organizar a mochila, por exemplo, reconhece-se que certos itens são sempre usados juntos.
3. **Abstração**:
   - *Definição*: Envolve focar nas informações essenciais e nos aspectos gerais do problema, ignorando detalhes irrelevantes no momento.
   - *Importância*: Reduz a complexidade do problema, tornando-o mais gerenciável, ajuda a manter o foco no que realmente importa e permite a generalização de soluções para outros problemas. No exemplo da mochila, a abstração é usada para focar apenas no que é essencial levar, ignorando o resto.
4. **Algoritmos**:
   - *Definição*: É o desenvolvimento de uma sequência de passos (instruções) bem definida para resolver um problema ou uma de suas partes.
   - *Importância*: Oferece uma solução clara e estruturada, permite a automação de tarefas e torna a solução replicável e adaptável. Para a mochila, um algoritmo seria a sequência criada para organizar os itens, como "colocar os itens maiores no fundo e os mais usados no topo".

3 Introdução aos Algoritmos

Algoritmo: É uma sequência lógica e bem definida de instruções ou passos que, quando seguidos, resolvem um problema específico ou realizam uma tarefa determinada. Vemos três principais formas de representar algoritmos:

3.1 Descrição Narrativa

- É a forma mais simples de representar um algoritmo, utilizando a linguagem natural (como o português) para descrever os passos em uma sequência lógica.
- É ideal para refletir sobre a sequência inicial de instruções. Exemplos práticos incluem uma receita de bolo, o cálculo da área de um retângulo e o cálculo da média de notas.

3.2 Fluxograma

- É uma representação gráfica do algoritmo que utiliza símbolos padronizados conectados por setas para indicar o fluxo de execução.
- Seu objetivo é ser visual e intuitivo, facilitando a compreensão do processo.
- Os símbolos básicos representam início/fim, entrada de dados, processamento, decisão e saída de dados.
- A apresentação também mostra exemplos de **Estruturas de Decisão** (condicionais SE/ENTÃO/SENÃO) e **Estruturas de Repetição** (loops PARA e ENQUANTO).

3.3 Pseudocódigo

- É uma linguagem intermediária entre a linguagem natural e uma linguagem de programação, usada para representar a lógica de um programa antes de sua implementação.
- Utiliza uma linguagem narrativa (como o Portugol) combinada com elementos de programação, mas sem uma sintaxe rígida, servindo como uma ferramenta de planejamento.
- A estrutura básica inclui as seções `ALGORITMO` (nome), `VARIAVEIS` (para declaração de dados, como `X: INTEIRO`), `INICIO`, o corpo com as instruções e `FIM`.

4 Manual de Pseudocódigo

O manual, feito pelo professor Ramon Mayor Martins, que pode ser acessado através **deste link** oferece um guia completo para a escrita de algoritmos em pseudocódigo, detalhando sua estrutura, sintaxe e principais comandos para o planejamento de programas de computador. Nesta sessão temos apenas um breve resumo do que pode ser encontrado lá!

4.1 Estrutura Básica de um Algoritmo

Um programa em pseudocódigo segue uma estrutura fundamental para garantir a organização e a clareza:

- `ALGORITMO nome_do_programa`: A primeira linha, que define o nome do algoritmo.
- `VARIAVEIS`: Bloco opcional onde todas as variáveis que serão utilizadas no programa são declaradas.
- `INICIO`: Palavra-chave que marca o começo do bloco de código principal, onde a lógica do programa é executada.
- `FIM`: Palavra-chave que indica o término do algoritmo.

4.2 Variáveis e Tipos de Dados

As variáveis representam dados que podem ser modificados durante a execução do programa e precisam ser declaradas com um tipo específico:

- `INTEIRO`: Para armazenar números inteiros, positivos ou negativos (ex: 10, -50).
- `REAL`: Para números com casas decimais (ex: 3.14, -2.5).
- `CARACTERE`: Para um único símbolo, letra ou número (ex: 'A', '7').
- `STRING`: Para armazenar sequências de caracteres, ou seja, texto (ex: "Olá, Mundo!").
- `LOGICO`: Para armazenar valores de verdadeiro ou falso (booleano).

4.3 Comandos de Entrada e Saída

Esses comandos permitem a interação do algoritmo com o usuário:

- `LEIA (nome_da_variavel)`: Pausa a execução do programa para que o usuário digite um valor, que é então armazenado na variável especificada.
- `ESCREVA`: Exibe informações na tela. Pode ser usado para mostrar textos fixos (entre aspas) ou o conteúdo de uma ou mais variáveis.

4.4 Operadores

São símbolos que executam operações sobre variáveis e valores:

- **Operadores Aritméticos**: Realizam cálculos matemáticos.
  - `+` (Adição), `-` (Subtração), `*` (Multiplicação), `/` (Divisão).
  - `%` (Resto da Divisão ou Módulo).
  - `RAIZ()` (Raiz Quadrada) e `^` (Potenciação).
- **Operadores Relacionais**: Realizam comparações entre dois valores, e o resultado é sempre um valor `LOGICO` (verdadeiro ou falso).
  - `>` (Maior que), `<` (Menor que), `>=` (Maior ou igual a), `<=` (Menor ou igual a).
  - `==` (Igual a) e `!=` (Diferente de).
- **Operadores Lógicos (Booleanos)**: Combinam duas ou mais expressões lógicas.
  - `E` (AND): Retorna verdadeiro somente se ambas as condições forem verdadeiras.
  - `OU` (OR): Retorna verdadeiro se pelo menos uma das condições for verdadeira.
  - `NAO` (NOT): Inverte o valor lógico de uma condição.

4.5 Estruturas de Controle

São blocos que direcionam o fluxo de execução do algoritmo, permitindo criar lógicas mais complexas:

- **Estruturas de Decisão**:
  - `SE-ENTAO-SENAO-FIMSE`: Utilizada para criar desvios condicionais. O código no bloco `ENTAO` é executado se a condição do `SE` for verdadeira; caso contrário, o código no bloco `SENAO` é executado. É possível aninhar estruturas `SE` umas dentro das outras.
  - `ESCOLHA-CASO-FIM-ESCOLHA`: Permite testar uma variável contra uma lista de valores (`CASO`). A cláusula `CASO CONTRARIO` é usada para tratar todos os outros casos não listados. É ideal para a criação de menus de opções.
- **Estruturas de Repetição (Loops)**:
  - `PARA-ATÉ-FIM-PARA`: Executa um bloco de código um número predefinido de vezes, geralmente utilizando uma variável contadora.
  - `ENQUANTO-FIM-ENQUANTO`: Repete um bloco de código enquanto uma determinada condição for verdadeira. O teste da condição é feito no início de cada iteração.

5 Introdução à Programação Orientada a Objetos (POO)

Por fim, temos uma introdução a Programação Orientada a Objetos (POO) como um paradigma de programação.

5.1 Conceitos Fundamentais da POO (Analogia com Lego)

- **Objeto**: É comparado a uma peça de Lego individual que pode ser usada para construir estruturas maiores.
- **Classe**: Representa o "molde"ou o projeto técnico detalhado de uma peça de Lego. Ela define as características (atributos) e as ações (métodos) que todos os objetos criados a partir dela terão. Cada peça física é uma **instância** de uma classe.
- **Atributos**: São as características das peças, definidas na classe, como cor, tamanho, forma e número de pinos.
- **Métodos**: São as ações que um objeto pode realizar, como `conectar()` e `desconectar()`.

5.2 Propósito da POO

A POO serve para:

- Facilitar o reuso e a manutenção do código.
- Melhorar a organização e a estrutura do código.
- Permitir a modelagem de problemas do mundo real de forma mais intuitiva.

5.3 Linguagem e Ferramentas

- **Java**: A linguagem de programação escolhida para o curso, por ser multiplataforma ("escreva uma vez, rode em qualquer lugar"), ter ampla adoção na indústria e uma grande comunidade. É adequada para aplicações empresariais, Android, web, mobile, entre outras.
- **Ferramentas**: O desenvolvimento foi feito utilizando o **Github Codespace** e também o **VSCode**.

6 Manual de Sintaxe da Linguagem Java

Este manual oferece um guia detalhado sobre a sintaxe fundamental da linguagem Java, cobrindo desde a estrutura básica de um programa até a manipulação de estruturas de dados como arrays e strings. A versão completa, feito pelo professor Ramon Mayor Martins, pode ser acessado através **deste link.**

6.1 Estrutura Fundamental e Tipos de Dados

- **Estrutura Básica**: Um programa Java é contido dentro de uma `class`. O ponto de entrada é o método `public static void main(String[] args)`, que é `public` para ser acessível externamente, `static` para ser executado sem criar um objeto da classe, e `void` porque não retorna nenhum valor.
- **Tipos Primitivos**: São os tipos de dados mais básicos.
  - *Inteiros*: `byte` (8 bits), `short` (16 bits), `int` (32 bits), e `long` (64 bits, sufixo L).
  - *Reais (Ponto Flutuante)*: `float` (32 bits, sufixo f, menor precisão) e `double` (64 bits, maior precisão).
  - *Outros*: `char` para um único caractere (entre aspas simples) e `boolean` para valores `true` ou `false`.
- **Constantes**: Variáveis cujo valor não pode ser alterado são declaradas com a palavra-chave `final`.
- **String**: Não é um tipo primitivo, mas sim uma classe para representar sequências de caracteres (texto).

6.2 Entrada e Saída de Dados (I/O)

- **Classe `Scanner`**: É a principal ferramenta para ler dados do usuário. Pertence ao pacote `java.util` e precisa ser importada. Um objeto `Scanner` é criado (`new Scanner(System.in)`) para ler a partir do teclado.
- **Métodos do `Scanner`**: Existem métodos específicos para ler cada tipo de dado, como `nextInt()`, `nextDouble()`, `next()` (lê uma palavra) e `nextLine()` (lê uma linha inteira).

6.3 Classes Wrapper

- São classes que "embrulham"os tipos primitivos em objetos, permitindo o uso de métodos úteis.
- Exemplos: `Integer` para `int`, `Double` para `double`, `Character` para `char`. Elas oferecem métodos estáticos como `Integer.toBinaryString()` e `Character.toUpperCase()`.

6.4 Operadores

- **Aritméticos**: `+`, `-`, `*`, `/`, e `%` (resto da divisão).
- **Relacionais**: Comparam valores e retornam `boolean`. São eles: `==`, `!=`, `>`, `<`, `>=`, `<=`.
- **Lógicos**: `&&` (E lógico), `||` (OU lógico) e `!` (NÃO lógico).
- **Classe `Math`**: Uma classe utilitária com métodos estáticos para operações matemáticas avançadas, como `Math.pow()`, `Math.sqrt()`, `Math.PI`, e `Math.random()`.

6.5 Estruturas de Controle

- **Estruturas de Decisão**:
  - `if-else`: Executa blocos de código com base em uma condição. Pode ser encadeado com `else if` ou aninhado.
  - `switch-case`: Testa uma variável contra múltiplos valores (`case`). `break` é usado para sair do bloco, e `default` trata os casos não previstos.
- **Estruturas de Repetição (Loops)**:
  - `for`: Ideal para repetir um bloco um número conhecido de vezes.
  - `while`: Repete um bloco enquanto uma condição for verdadeira (teste no início).
  - `do-while`: Similar ao `while`, mas o teste da condição é feito no final.

6.6 Arrays

- **Definição**: Estrutura que armazena uma coleção de elementos de tamanho fixo e do mesmo tipo. O acesso é feito por um índice que começa em zero.
- **Propriedade `.length`**: Fornece o número de elementos no array.
- **Arrays Multidimensionais**: Usados para criar matrizes (arrays de arrays).
- **Classe `Arrays`**: Classe utilitária com métodos estáticos para manipular arrays, como `Arrays.sort()`, `Arrays.binarySearch()`, e `Arrays.toString()`.

6.7 Manipulação de Strings

- **Imutabilidade**: Strings em Java são objetos imutáveis; qualquer alteração cria uma nova String.
- **Principais Métodos**: A classe `String` oferece diversos métodos úteis:
  - `+` ou `.concat()`: Para juntar strings.
  - `.length()`: Para obter o tamanho.
  - `.substring()`: Para extrair parte de uma string.
  - `.equals()` e `.equalsIgnoreCase()`: Para comparar o conteúdo.

- `.compareTo()`: Para comparar a ordem lexicográfica.
- `.contains()`: Para verificar a existência de uma substring.

7 Referências

[1] GOOGLE DRIVE. Pensamento Computacional e Algorítmos. Disponível em: https://drive.google.com/drive/folders/1KD8tLxErxiQujd1Xr8ehZCbPzx0SPczv?usp=drive_link. Acesso em: 20 ago. 2025.

[2] MARTINS, Ramon Mayor. *Manual Java - Sintaxe*. Google Docs, 2024. Disponível em: https://docs.google.com/document/d/1G3ILrjtuUFyg_8eSQ3ZOIOP__UqQWILluZvzC4ZKXH4/edit?usp=sharing. Acesso em: 20 ago. 2025.

[3] MARTINS, Ramon Mayor. *Manual Pseudocódigo-cb*. Google Docs, 2025. Disponível em: https://docs.google.com/document/d/1d73HQra7Tljce-U89eE3fGZUwSoMHN2EYX8NL2M5FDs/edit?usp=sharing. Acesso em: 20 ago. 2025.